3 Roteiro 03 – Construindo uma API com Flask a partir de Classes Python

Objetivos

- Compreender como transformar lógica orientada a objetos em uma API Web;
- Estruturar uma aplicação Flask de forma organizada;
- Trabalhar com JSON como formato de resposta;
- Testar requisições HTTP com navegador, `curl` e Postman;
- Aplicar versionamento adequado no desenvolvimento backend.

Ferramentas Necessárias

Python 3.11+, Flask, Postman (ou Insomnia), Git, navegador web e terminal.

Introdução

No roteiro anterior desenvolvemos um sistema utilizando Programação Orientada a Objetos. Agora iremos transformar essa lógica em uma API Web utilizando Flask, entendendo como um backend organiza regras de negócio e disponibiliza dados via HTTP.

Além de construir a aplicação, também veremos como testar suas rotas corretamente. Esse ponto é importante porque diferentes métodos HTTP produzem comportamentos diferentes: por exemplo, o navegador normalmente faz requisições **GET**, enquanto ferramentas como `curl` e Postman permitem testar também requisições **POST**.

Fundamentação Teórica

Uma **API Web** permite que sistemas se comuniquem por meio de requisições HTTP.

O Flask é um microframework em Python que possibilita:

- Criar rotas HTTP;
- Retornar dados em formato JSON;
- Receber dados enviados pelo cliente;
- Organizar regras de negócio em aplicações web.

JSON é o formato padrão de comunicação entre frontend e backend.
Também é importante compreender a diferença entre alguns métodos HTTP:

- **GET**: usado para consultar informações;
- **POST**: usado para enviar dados e criar novos registros;
- **PUT/PATCH**: usados para atualizar dados;
- **DELETE**: usado para remover dados.

Quando uma rota não aceita o método enviado pelo cliente, o servidor pode retornar o erro **405 Method Not Allowed**. Por isso, ao testar uma API, precisamos confirmar se estamos usando o método correto para cada rota.

Parte 1 – Preparando o Projeto

Antes de escrever qualquer código, precisamos organizar o ambiente da aplicação. Um projeto backend deve possuir:

- Pasta dedicada;
- Ambiente virtual isolado;
- Dependências instaladas corretamente;
- Arquivos organizados por responsabilidade.

Nesta etapa iremos:

- Criar a pasta do projeto;
- Criar um ambiente virtual Python;
- Instalar o Flask;
- Criar os arquivos iniciais;
- Inicializar o versionamento com Git.

Execute os comandos abaixo no terminal:

```
mkdir api-alunos
cd api-alunos
python -m venv venv
source venv/bin/activate
pip install flask
touch app.py
touch aluno.py
git init
```

O que cada comando faz:

- `mkdir api-alunos` → cria a pasta do projeto;
- `cd api-alunos` → entra na pasta criada;
- `python -m venv venv` → cria ambiente virtual isolado;
- `source venv/bin/activate` → ativa o ambiente virtual;
- `pip install flask` → instala o framework Flask;
- `touch app.py` → cria arquivo principal da aplicação;
- `touch aluno.py` → cria arquivo da classe de modelo;
- `git init` → inicia controle de versão.

Por que usamos ambiente virtual?
Para evitar conflitos entre bibliotecas instaladas no sistema e manter o projeto isolado.

Parte 2 – Reutilizando a Classe Aluno

No roteiro anterior desenvolvemos a classe `Aluno` utilizando Programação Orientada a Objetos. Agora iremos reutilizar essa mesma classe dentro de uma aplicação web. Isso demonstra uma prática profissional importante: **separar regras de negócio da camada web**.
O arquivo `aluno.py` será responsável apenas pela lógica do sistema, enquanto o Flask ficará responsável pelas rotas HTTP.
Abra o arquivo `aluno.py` e insira o código abaixo:

```
class Aluno:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
        self.notas = []

    def adicionar_nota(self, nota):
        self.notas.append(nota)

    def calcular_media(self):
        if not self.notas:
            return 0
        return sum(self.notas) / len(self.notas)

    def aprovado(self):
        return self.calcular_media() >= 7

    def to_dict(self):
        return {
            "nome": self.nome,
            "idade": self.idade,
            "media": self.calcular_media(),
            "aprovado": self.aprovado()
        }
```

O que está acontecendo neste código:

- O método `__init__` define os atributos do objeto;
- `adicionar_nota()` permite modificar o estado do aluno;
- `calcular_media()` encapsula a regra de cálculo;

- `aprovado()` aplica a regra de negócio;
- `to_dict()` converte o objeto para um formato compatível com JSON.

Por que criamos o método `to_dict()`?
O Flask não consegue enviar diretamente um objeto Python como resposta HTTP. Ele precisa de um formato serializável (como dicionário), que depois será convertido em JSON pelo método `jsonify()`.
Portanto, o fluxo será:
Objeto Python → Dicionário → JSON → Resposta HTTP
Esse é um dos conceitos mais importantes na construção de APIs.

Parte 3 – Criando a API com Flask

Agora iremos criar a aplicação web utilizando Flask. Nesta etapa conectaremos:

- A classe `Aluno` (regra de negócio);
- A lista de armazenamento em memória;
- Uma rota HTTP que retorna dados em formato JSON.

O objetivo é compreender como o backend disponibiliza dados para outros sistemas por meio de requisições HTTP.

Abra o arquivo `app.py` e insira o código abaixo:

```
from flask import Flask, jsonify, request
from aluno import Aluno

app = Flask(__name__)

alunos = []

aluno1 = Aluno("João", 17)
aluno1.adicionar_nota(6)
aluno1.adicionar_nota(7)
aluno1.adicionar_nota(8)

aluno2 = Aluno("Maria", 16)
aluno2.adicionar_nota(5)
aluno2.adicionar_nota(6)
aluno2.adicionar_nota(7)

alunos.append(aluno1)
alunos.append(aluno2)

@app.route("/alunos", methods=["GET"])
def listar_alunos():
    return jsonify([aluno.to_dict() for aluno in alunos])

@app.route("/alunos", methods=["POST"])
def adicionar_aluno():
    dados = request.get_json()
    novo_aluno = Aluno(dados["nome"], dados["idade"])
    alunos.append(novo_aluno)
    return jsonify(novo_aluno.to_dict()), 201

if __name__ == "__main__":
    app.run(debug=True)
```

O que está acontecendo neste código:

- `Flask(__name__)` cria a aplicação web;

- A lista `alunos` simula um banco de dados em memória;
- Criamos dois objetos da classe `Aluno`;
- O decorador `@app.route("/alunos", methods=["GET"])` define uma rota de consulta;
- O decorador `@app.route("/alunos", methods=["POST"])` define uma rota de criação;
- Quando alguém acessa `/alunos` com GET, a função `listar_alunos()` é executada;
- Quando alguém envia dados para `/alunos` com POST, a função `adicionar_aluno()` é executada;
- A list comprehension converte cada objeto em dicionário;
- `jsonify()` transforma os dicionários em JSON válido;
- `debug=True` ativa recarregamento automático e mensagens de erro detalhadas.

Importante: Objetos Python não podem ser enviados diretamente pela web. Por isso utilizamos o método `to_dict()` para convertê-los antes de aplicar `jsonify()`.
Outro ponto importante é que cada rota aceita métodos HTTP específicos. Se tentarmos acessar uma rota usando um método não permitido, o Flask poderá retornar **405 Method Not Allowed**. Por exemplo, abrir uma URL no navegador normalmente envia um **GET**; isso não substitui um teste de **POST**.
O fluxo da requisição GET será:
Cliente → Flask → Função da rota → Objeto → Dicionário → JSON → Resposta HTTP

Execute a aplicação:

```
python app.py
```

Você verá no terminal que o servidor foi iniciado.

Teste no navegador:

`http://127.0.0.1:5000/alunos`

Se tudo estiver correto, será exibido um JSON contendo os alunos criados.
Observe também o terminal: Cada requisição realizada aparecerá registrada no log do servidor.

Teste também com `curl`:

```
curl http://127.0.0.1:5000/alunos
```

Esse comando faz uma requisição **GET** para a rota `/alunos`. A resposta esperada é uma lista JSON com os alunos cadastrados.

Parte 4 – Criando Rota POST

Até agora nossa API apenas retornava dados (método GET). Agora iremos permitir que o cliente envie informações para o servidor utilizando o método HTTP **POST**.
Essa etapa é importante porque demonstra como o backend:

- Recebe dados em formato JSON;
- Converte esses dados para objetos Python;
- Atualiza o estado da aplicação;
- Retorna uma resposta adequada ao cliente.

Observe que a rota POST já está no arquivo `app.py`:

```
@app.route("/alunos", methods=["POST"])
def adicionar_aluno():
    dados = request.get_json()
    novo_aluno = Aluno(dados["nome"], dados["idade"])
    alunos.append(novo_aluno)
    return jsonify(novo_aluno.to_dict()), 201
```

O que está acontecendo neste código:

- `request.get_json()` captura o JSON enviado pelo cliente;
- O dicionário recebido é armazenado na variável `dados`;
- Criamos um novo objeto da classe `Aluno` com as informações recebidas;
- O objeto é adicionado à lista `alunos`;
- A resposta retorna o novo aluno convertido para JSON;
- O código `201` indica que um recurso foi criado com sucesso.

Fluxo da requisição POST:
Cliente → Envia JSON → Flask recebe → Cria objeto → Atualiza lista → Retorna JSON + 201

Testando com `curl`
No terminal, execute:

```
curl -X POST http://127.0.0.1:5000/alunos \
  -H "Content-Type: application/json" \
  -d '{"nome":"Carlos","idade":18}'
```

Explicando o comando:

- `-X POST` define o método HTTP como POST;
- `-H "Content-Type: application/json"` informa ao servidor que estamos enviando JSON;
- `-d '{...}'` envia os dados no corpo da requisição.

Se estiver correto, a resposta deverá retornar:

- Status **201 Created**;
- JSON contendo os dados do novo aluno.

Testando com Postman
Configure no Postman:

- Método: POST
- URL: `http://127.0.0.1:5000/alunos`
- Aba `Body`
- Opção `raw`
- Tipo: `JSON`

Envie o seguinte conteúdo:

```
{
  "nome": "Carlos",
  "idade": 18
}
```

Se estiver correto, a resposta deverá retornar:

- Status: **201 Created**

- JSON contendo os dados do novo aluno

Conferindo o resultado após o POST

Depois de enviar o POST, execute novamente um GET para verificar se o novo aluno foi adicionado. Com `curl`:

```
curl http://127.0.0.1:5000/alunos
```

Ou abra novamente no navegador:

`http://127.0.0.1:5000/alunos`

Isso demonstra que a API está modificando o estado interno da aplicação.

Atenção ao erro 405 Method Not Allowed

Se aparecer a mensagem **Method Not Allowed**, verifique:

- Se a rota está correta: `/alunos`;
- Se o método usado é realmente **POST**;
- Se o servidor Flask está em execução;
- Se o corpo da requisição foi enviado em formato JSON;
- Se você não está tentando testar POST apenas abrindo a URL no navegador.

Lembre-se: o navegador normalmente envia **GET**. Para testar **POST**, o ideal é usar `curl`, Postman ou Insomnia.

Versionamento

Em desenvolvimento profissional, cada evolução deve ser registrada no controle de versão.
Após implementar e testar a rota POST, execute:

```
git add .
git commit -m "Implementa rota GET e POST para API de alunos"
```

Evite mensagens genéricas como `"update"` ou `"teste"`.
Commits devem ser:

- Claros;
- Específicos;
- Pequenos e frequentes.

Isso facilita manutenção futura e colaboração em equipe.

Entrega

O estudante deverá entregar:

- Aplicação Flask funcional;
- Classe Aluno separada em arquivo próprio;
- Rota GET e POST funcionando;
- Testes realizados com navegador, `curl` ou Postman;
- Retorno estruturado em JSON;
- Histórico de commits organizados;
- README explicando como executar a aplicação.